

Reuse, but Verify: Certified Maintenance of LLM-Computed Table Cells under Prompt Edits

Arjun Lohan
Independent Researcher
United States
arjunlohan7@gmail.com
lohan@usc.edu

Abstract

AI-native tables let users define columns in natural language: each cell is computed by a language model from a versioned prompt over its row. When the prompt is edited, deployed systems face a choice: recompute every cell, which is expensive and rate-limited, or silently reuse stale ones, which incurs unbounded error. We present sIVM, a maintenance layer that certifies which cached cells may be reused after a prompt edit, bounding the expected false-reuse rate of each certified stratum by a user budget α with probability at least $1 - \delta$. sIVM freezes cached-value strata, samples adaptively on a doubling look schedule, and bounds each stratum’s flip rate with empirical-Bernstein confidence bounds; a strict mode certifies at a deflated threshold to bound the error of the reused cells alone. We also establish an impossibility floor: a model’s rate of disagreement with itself under the unedited prompt floors the certifiable budget. On two public corpora (89,184 Stack Overflow developer profiles; 210,250 Djinni candidate CVs) with 32,000+ labeled cells and a production table system, a formatting-only edit certifies 82.6% call savings at $\alpha=0.2$, the loose end of a measured savings–budget frontier, and at deployment scale 225 oracle calls certify reuse of 81,289 of 89,184 cells (91.1% fewer model calls). In every seeded sweep and sampling replication, and in 396,000 known-rate calibration trials, no certified stratum’s true flip rate exceeded its budget. We release the system, the labels, and a benchmark of versioned prompt-edit pairs.

1 Introduction

Interactive AI tables (data-enrichment sheets, recruiter search tables, spreadsheet copilots) turn a natural-language prompt into a *materialized view*: a column $V = (\pi, M, \theta)$ with prompt π , model M , and pinned decode parameters θ , materialized as cells $c_i = f_V(r_i)$ over rows $r_1, \dots, r_n$. These views are edited constantly: users iterate on prompts the way they iterate on spreadsheet formulas. Every edit invalidates the cache by construction, and today’s systems invalidate it wholesale: an edit to an 89,184-row column triggers 89,184 fresh model calls, or the user silently keeps stale values with no bound on how wrong they are.

Recent semantic data systems bound the error of *one-shot* query execution against a gold model (cascades, semantic operators, approximate processing with guarantees), and verified semantic caches bound reuse under a *fixed* definition. Neither answers the maintenance question an interactive table actually poses: *after*

this specific edit $\pi \rightarrow \pi'$, which cached cells are still valid, and with what guarantee?

1.1 Why intuition fails here

Three measurements reframe the problem (§5). First, LLM cells are stochastic even at temperature zero: the same prompt on the same row disagrees with itself 5.0% of the time on our boolean workload (within-version draw pairs on $n=2,000$; an earlier $n=200$ two-draw probe read 3.0%) and 9.5% at default temperature, so “did the edit change this cell?” is only meaningful relative to a noise floor. Second, edit direction is not monotone: under a strictly scope-*widening* edit, 45.7% of cached-TRUE cells flipped ([39.0, 52.7] over 199 such cells), refuting the folk lemma that widening preserves positives. Third, surface-level edit classes do not predict semantic impact. On the same column, a meaning-preserving synonym paraphrase flipped 8.90% of cells where a formatting-only edit flipped 6.3%. On the same CV column, a formatting-only edit flipped 12.30% against 12.45% for a deliberate criteria change, rates that are indistinguishable. Neither pair orders the way the edits’ labels suggest. Both comparisons hold the column fixed deliberately: across columns, the column’s own stability dominates the edit.

Consequently, sound reuse cannot be rule-based. It must be *measured*: sample a few fresh cells, statistically bound the flip rate of the rest, and reuse only what clears the user’s error budget.

1.2 Contributions

- (1) **Problem formalization** (§2): certified cross-version reuse for stochastic prompt-defined views, with a pair of estimands that are well-defined under decode pinning (expected false-reuse over the cells a certified stratum presents, which we certify by default, and over the reused cells alone, which the strict mode certifies at a deflated threshold), and its impossibility floor (the intrinsic flip rate).
- (2) **The sIVM procedure** (§3): frozen cached-value stratification (with optional interaction refinement), grid-adaptive sampling with a doubling look schedule (validity under peeking by δ -splitting), futility stopping, per-stratum empirical-Bernstein certification, and two guarantee targets (presented cells; strict reuse set); any frozen stratifier affects power only, never validity.
- (3) **A production implementation** (§4) inside a working AI-table system (per-cell runner, content-hash cache, cost ledger, certificates surfaced in the UI), not a simulation harness.

- (4) **An empirical anatomy** (§5) on two real corpora with 32,000+ labeled cells: the noise floor, the edit-class savings curve, the α -cost frontier, minority-stratum volatility and its vote-of- k diagnosis, and a stratifier ablation showing that refinement is validity-free but costs power on this benchmark. Every certified sweep held the bound it certified.
- (5) **A reusable benchmark** (§6): versioned prompt-edit pairs with full ground-truth flip labels over public, redistributable corpora (ODbL and MIT), enabling like-for-like evaluation of future maintenance systems.

2 Problem

Views and cells. An AI column is a view $V = (\pi, M, \theta)$: a prompt template π with typed output contract, a model M , and pinned decode parameters θ . For row r_i , the bound prompt $\pi(r_i)$ interpolates only the fields $\beta(\pi)$ that π references; the stored cell c_i is *one draw* from the output distribution $f_V(r_i)$. Even at temperature zero this distribution is not degenerate: providers are nondeterministic across calls, so every quantity below is defined over draw randomness.

The maintenance event. The user edits the definition, $V \rightarrow V' = (\pi', M, \theta)$. For each cached cell define the flip indicator $F_i = \mathbb{1}[c_i \neq X'_i]$, where $X'_i \sim f_{V'}(r_i)$ is a fresh draw under the new definition and $\equiv$ is the column’s equivalence relation (exact match for the boolean and select columns studied here). F_i is Bernoulli with unknown mean $p_i = \Pr[c_i \neq X'_i]$. Note that p_i deliberately folds together the edit’s semantic effect and decode noise on both sides: it answers the operational question of whether recomputing cell i now would return a value different from the one shown.

Estimand and guarantee. A maintenance policy partitions the cached cells into a sampled set $\mathcal{S}$ (freshly recomputed), a reuse set $\mathcal{R}$ (stale values kept), and a recompute set. Two error rates matter, and they differ. The *presented-cells* rate of a certified stratum j is

$$\text{FR}_j^{\text{pres}} = \frac{1}{n_j} \sum_{i \in j \setminus \mathcal{S}} F_i :$$

of all n_j cells the user now sees from that stratum, the fraction that are stale-and-wrong (sampled cells are fresh and contribute zero). The *reuse-set* rate

$$\text{FR}_j^{\text{reuse}} = \frac{1}{n_j - m_j} \sum_{i \in j \setminus \mathcal{S}} F_i$$

normalizes by the $n_j - m_j$ reused cells alone and is never smaller. Certifying the whole-stratum mean at α bounds $\text{FR}_j^{\text{pres}}$ in expectation but not $\text{FR}_j^{\text{reuse}}$: the remainder is adversely selected, because certification happens exactly when the sample looked clean. sIVM therefore certifies the presented-cells estimand by default, and offers a strict reuse-set mode that certifies at the deflated threshold $\alpha(n_j - m_j)/n_j$, which implies $\mathbb{E} \text{FR}_j^{\text{reuse}} \leq \alpha$ by conservation of flips. Both realized rates are reported for every experiment (Table 1). All guarantees are per-stratum and therefore hold simultaneously for every union of certified strata, not only the aggregate (§5, B2, shows why aggregate-only certification is operationally misleading).

Assumptions.

Assumption 1 (Pinned snapshot). M and θ are fixed during certification; cell draws are distributed as $f_{V'}(r_i)$ throughout the window and are independent *across rows* (provider-side batching that correlates concurrent responses would violate this; our runner issues independent requests). Certificates are scoped to the snapshot; a provider-side model change invalidates them (change-of-world maintenance is complementary work).

Assumption 2 (Frozen stratification). The stratifier is a deterministic function of the cache, the edit, and row content, fixed before any oracle call. Any such stratifier preserves validity; its quality affects only power (§5, “The stratification hierarchy”).

Assumption 3 (Uniform within-stratum sampling). Sampled rows are drawn by a seeded uniform shuffle within each stratum, without replacement. We apply i.i.d. empirical-Bernstein bounds to these draws and do *not* prove that they remain valid under without-replacement sampling; this is an assumption, and §3 states what replacing it would cost.

No monotonicity shortcuts. Folk reasoning suggests one-sided safety: a scope-widening edit should preserve TRUE cells, so cached TRUEs could be reused for free. Our measurements refute the unconditional form: under a widening edit, 45.7% of cached-TRUE cells flipped, driven largely by boundary instability, not semantics. Edit direction therefore enters sIVM only as a stratification hint; any directional certificate must be *conditional* on a measured violation rate, never assumed.

The impossibility floor is per-stratum. Let s_j be stratum j ’s self-flip rate: the probability that two independent draws of the *same* cell disagree, *conditional on the cached draw that put the cell in stratum j* . For the identity edit $\pi' = \pi$, p_j equals s_j , so no procedure whose oracle is a single fresh draw can certify stratum j at $\alpha < s_j$: the certifiable budget is floored by model stability. Two scoping notes apply. The conditioning is not incidental. Because membership is decided by one draw, a minority-value stratum is selected partly *for* having flipped that way, so s_j is a property of the (cache, stratifier) pair, not of the column, and recomputing the cache with vote-of- k at write time lowers it (§5, “Minority-stratum anatomy”). And the edit-independence is derived for the identity edit only; an edit that sharpens the output distribution can drive p_j below s_j , so we state edit-independence as an empirical regularity on our workloads, not as a bound.

The floors are unequal, and the column-level average describes neither stratum. On our boolean column the cached-FALSE stratum has $s=3.1\%$ (95% row-cluster bootstrap [2.5, 3.7]) while the cached-TRUE stratum has $s=24.1\%$ ([19.6, 28.8]), against a pooled 5.0% that is their mixture. Rows are bucketed by the stored cached value the certifier itself stratifies on, and the interval resamples rows, since draw-pairs from one row are dependent. Three otherwise separate observations follow from this single measurement. The FALSE stratum is the only one that certifies at all, and it is certifiable in principle down to budgets near its own floor; in practice at $\alpha=0.05$ it certifies in only 0.8% of replications (Table 1) because power, not the floor, becomes binding there. The TRUE stratum is never certified at any budget we tried: its floor exceeds every

budget we ran. Its size sets the deployment-scale savings ceiling: with 7,715 of 89,184 cells cached TRUE, reuse cannot exceed 91.3%. The ceiling belongs to *this* cache, and a cache built with vote-of- k at write time would hand the certifier a smaller stratum. And any method that reuses cells from that stratum inherits an error rate at or above its floor no matter what aggregate target it was given, which is what we observe when we run one (§5, B2'). Column-level stability is the wrong unit; the certifier's stratification recovers the right one, and a standing refusal on a stratum is a stability diagnosis of that slice of the column.

3 The sIVM procedure

Given the edit $\pi \rightarrow \pi'$ and cached cells for the affected rows (Figure 1):

- (1) **Diff.** Compute a word-level LCS diff of the prompts; collect content terms of added/removed spans (the *delta*).
- (2) **Stratify (frozen).** Partition rows by cached value. We adopt the skeleton of proxy-certified selection [11] deliberately here: with a binary proxy this step is their reduction, transported from one-shot selection to maintenance. What we change is the guarantee it carries, from one aggregate bound over the selected set to a bound per stratum, which is the only difference §5 (B2') isolates and the only one that protects a subgroup whose floor differs from the column's. Optionally refine each value stratum by an interaction bucket (lexical delta-term overlap or embedding cosine between the delta text and the row's bound content, tertile-bucketed): a validity-free refinement that pays only when within-value flip risk is heterogeneous, and otherwise costs power through the δ/K split (§5).
- (3) **Schedule.** For stratum j of size n_j , fix a doubling look schedule $L_j = (45, 90, 180, \dots)$ capped at n_j and sized so the final look can certify a zero-flip stratum; split the stratum's error budget $\delta_j = \delta/K$ evenly across looks.
- (4) **Sample and test.** Draw the stratum's seeded shuffle once. At each look t , reveal fresh draws for the prefix, compute the Maurer–Pontil [18] empirical-Bernstein upper confidence bound $u_{j,t}$ on the stratum flip rate at level $\delta_j/|L_j|$; *certify and stop* if $u_{j,t} \leq \alpha$; *stop for futility* if the empirical rate alone exceeds α .
- (5) **Apply.** Reuse all unsampled rows of certified strata (recording certificate provenance per cell); recompute the rest. Sampled rows are already fresh.

THEOREM 3.1 (VALIDITY UNDER PEEKING). *Let K be the number of strata produced by the frozen stratifier and $\delta_j = \delta/K$. Under Assumptions 1–3, with probability at least $1 - \delta$ over sampling and oracle draws, every certified stratum j has true flip rate $p_j \leq \alpha$; consequently $\text{FR}_j^{\text{pres}} \leq \alpha$ in expectation for every certified stratum and every union of certified strata (the empty certification is vacuously valid). In reuse-set mode, where certification at sample size m_j requires $u_{j,t} \leq \alpha(n_j - m_j)/n_j$, the same event further implies $\text{E}\text{FR}_j^{\text{reuse}} \leq \alpha$ for every certified stratum.*

PROOF SKETCH. Fix stratum j with $p_j > \alpha$ (resp. $p_j > \alpha(n_j - m_j)/n_j$ at every look, in reuse-set mode). Certifying requires some look t whose empirical-Bernstein upper bound falls

below the true rate, an event of probability at most $\delta_j/|L_j|$ per look; union bounds over the $|L_j|$ looks and K strata give δ . The futility rule only stops sampling, never certifies, so it cannot increase the error probability. On the good event: the expected presented-cells error of stratum j is at most its whole-stratum rate $p_j \leq \alpha$, and any size-weighted union of certified strata inherits the bound. For reuse-set mode, total expected flips in the stratum are $n_j p_j \leq n_j \alpha(n_j - m_j)/n_j$, and at worst all of them fall in the remainder, so the remainder mean is at most α in expectation. Sampling within a stratum is a uniform without-replacement draw, while the empirical-Bernstein bound we apply is stated for i.i.d. draws. We do not prove that the i.i.d. bound dominates in this regime, and Assumption 3 carries that gap explicitly; sampling fractions in our pinned runs reach 81%, well outside any small-fraction argument. Finite-population and betting bounds [2, 25] are the principled replacement, and §5 measures both inside the same pinned procedure: a betting confidence sequence is tighter on this workload and certifies more configurations (9 against 7 in the main seeded run), while the without-replacement bound performs on par with Maurer–Pontil (8). Our betting construction is the i.i.d. hedged capital process; it purchases power, not without-replacement validity, so the gap this assumption names stays open. A recent ablation of nine bound families for selective prediction under risk control catalogues the same kind of regime dependence [3]. Closing this gap properly means either proving the i.i.d. bound sufficient in this regime or paying that power cost, and we do neither here. Our calibration study (§5) measures the procedure as deployed, at every budget at which the paper reports a certification ($\alpha \in \{0.05, 0.1, 0.2\}$) and with 54 configurations planted just above the budget, and finds no unsafe certification across 396,000 trials. That is evidence for the regimes we ran, not a proof for all of them. $\square$

Power. A clean stratum certifies at the zero-flip minimum $n^*(\alpha, \delta') = \lceil 7 \ln(2/\delta') / (3\alpha) \rceil + 1$ (the variance term vanishes), e.g. $n^* = 257$ at $\alpha = 0.05$ under the pinned procedure ($K=2$, six looks). That is the *fixed-plan* minimum, and the doubling schedule cannot stop on it: the first look at or above n^* is the one it takes, so a clean stratum actually pays 360 draws in calibration. Either way, savings require strata much larger than n^* , i.e. large tables, which is precisely the deployment regime. Near the frontier, certifying a stratum with true rate p requires roughly $n \gtrsim \frac{2p(1-p)\ln(2/\delta')}{(\alpha-p)^2}$ samples, which explains the measured collapse of savings as $\alpha \downarrow p$ (Table 1, SO formatting: 82.6% $\rightarrow$ 63.6% $\rightarrow$ 0.1% mean savings $\rightarrow$ refusal as α tightens 0.2 $\rightarrow$ 0.02).

4 Implementation

sIVM runs inside *lore*, a working AI-table system: a natural-language search bar compiles to a versioned filter specification executed identically on MySQL and Elasticsearch (conformance-tested for count, page, sort, aggregation, and hydration parity on 89,184 rows); AI columns are per-cell jobs with typed structured outputs, bounded concurrency, retry, and a content-hash cache keyed on *only the bound fields* $\beta(\pi)$, so unrelated row changes never invalidate the cache and content-duplicate rows share one computation. Every run passes a pre-run cost gate and lands

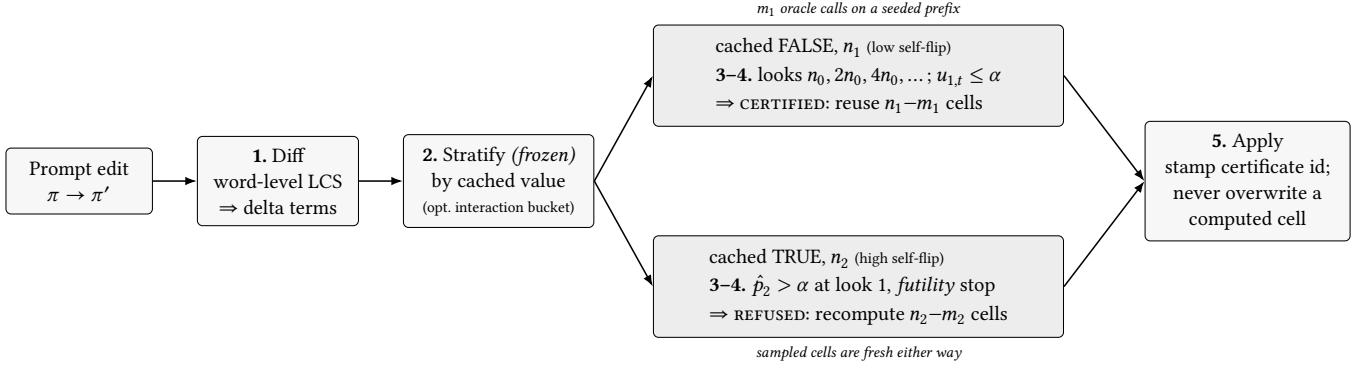


Figure 1: The procedure on the configuration our experiments pin. Stratification is frozen before any oracle call, so it moves power but never validity (Assumption 2); only steps 3–4 spend model calls, and they stop on evidence about a stratum’s *rate*, which is independent of the stratum’s size. The two lanes are the outcome we measure throughout: the stable stratum certifies and is reused under a certificate, while the volatile one is refused at every budget we tried and recomputed. A refusal is a stability diagnosis of that slice of the column (§2), reached at the cost of one look instead of a full recomputation.

in a ledger; prompt edits create immutable versions (the sIVM substrate); certificates persist with full strata evidence, and reused cells carry visible provenance in the UI. Decode is pinned ($T=0$) for all cell computation per Assumption 1.

Storage model. Cells are keyed by (column, row, prompt version), so versions of a column *coexist* instead of replacing one another, and a secondary index on (column, version, model, content hash) serves the cache. This is the one design decision sIVM actually depends on. Systems that key a cell by row alone, or that hash the prompt into the cache key, destroy the old materialization at the moment of the edit, and the question this paper asks cannot then be posed: there is nothing left to certify. Keeping versions side by side turns “which cached cells still transfer?” into a join between two versions of one table, which is why the certifier is a query over the store, not a service beside it.

Certificates are records, not decisions. A certificate persists the budget α , the prompt delta, the per-stratum evidence (size, cells sampled, bound attained, certified or refused), and the sampled, reused, and recompute counts. It is therefore auditable after the fact: the realized-error audits in §5 are recomputed from the certificate’s own persisted reuse set by a script that shares no code with the certifier. Applying a certificate is a single `INSERT ... SELECT` that carries certified values forward under the new version and stamps the certificate identifier on each one.

That insert carries an anti-join against the target version, so reuse is strictly *additive*: a certificate can supply a value where none was computed, and can never overwrite one that was. An earlier implementation used an upsert, which is the obvious way to write it and is silently destructive: applying a certificate after a partial recompute reverted freshly computed cells to the stale values the certificate had been asked to reuse. The bug is invisible in aggregate error, because a reverted cell and a reused cell are the same row with the same value; it surfaces only if something outside the certifier knows what that cell should have said. Any system that maintains stochastic cells across versions has this hazard, and the guard belongs in the write path, not in code review.

5 Experiments

Baselines. Three comparisons isolate what sIVM adds. (B1) *Verified-cache similarity has no cross-version signal*: the per-row similarity between the v1-bound and v2-bound prompts, the quantity a verified semantic cache thresholds, predicts cell flips at chance (AUC 0.468 formatting, 0.499 widening; on the formatting edit the score’s entire range is 0.96–0.99, because shared row text dominates). Adding “a version key” to such a cache therefore cannot produce a sound cross-version policy. Our interaction score carries real signal exactly when the edit is semantic (AUC 0.669 on widening) and none above chance when flips are noise-driven (AUC 0.408 on formatting), which is why the *cached-value* dimension of stratification carries the formatting case. (B2) *Aggregate-only certification (no strata)*, run with the *same* schedule and budgets as the pinned procedure, is valid for its aggregate estimand and silently corrupts subgroups. On the formatting edit it certifies at $\alpha=0.1$ (28.0% savings, 6.07% aggregate error) while the reused cached-TRUE subgroup carries 45.2% error, and at $\alpha=0.2$ it certifies 91.0% at 5.99% aggregate with 36.1% subgroup error; on the widening edit it refuses at $\alpha=0.1$ and certifies 64.0% at $\alpha=0.2$ with 44.1% subgroup error. Per-stratum bounds are what make the guarantee mean something for every user-visible slice of the table, and the effect is not an artifact of giving the baseline a shorter schedule. (B2’) *A SUPG-style competitor, matched.* The closest existing method is proxy-certified selection [11]: score rows with a cheap proxy, spend an oracle budget labelling a sample, threshold the proxy so the selected set’s error clears the target. We transported it to maintenance with the proxy the reduction in §3 names, the cached value itself, and matched it to sIVM on every axis we could: identical oracle budget, identical labels, identical error target, the same total error budget δ , each split across the number of tests its own procedure performs (sIVM across strata and looks, $\delta/12$ per test; the competitor across its candidate thresholds, $\delta/3$), and the same reported quantity (error among reused cells, not the presented-cells rate sIVM certifies).

Table 1: The pinned primary procedure (cached-value strata; adaptive doubling looks $n_0=45$, six looks; $\delta=0.1$; presented-cells estimand) on all five edit pairs, $n=2,000$ each at temperature 0. FR_{pres}/FR_{reuse} are realized false-reuse over presented cells and over the reuse set alone for the main seed; savings (with 95% percentile intervals) and certification rate are over $B=1,000$ sampling replications on frozen ground-truth labels; the reuse-set violation rate is *conditional on certifying*, i.e. its denominator is the replications that issued a certificate, not B ; FR_{reuse} carries a Wilson interval. Reuse-set violation rates are 0.0% in every configuration except SO formatting at $\alpha=0.05$, a frontier-straddling cell discussed below. The table is generated mechanically from the result files; earlier exploratory sweeps with other stratifiers and schedules are reported as ablations, not headline numbers.

Edit	α	flip	oracle	reused	FR_{pres}	FR_{reuse} [95% CI]	savings % [95% PI]	cert.	exceed.
SO formatting-only	0.02	6.3%	225	0	–	–	0.0 [0,0]	0%	–
SO formatting-only	0.05	6.3%	1485	0	–	–	0.1 [0,0]	1%	100.0%
SO formatting-only	0.10	6.3%	765	1094	1.65%	2.74 [1.9,3.9]	63.6 [55,73]	99%	0.0%
SO formatting-only	0.20	6.3%	135	1724	3.20%	3.36 [2.6,4.3]	82.6 [82,86]	100%	0.0%
SO synonym rewording	0.02	8.9%	90	0	–	–	0.0 [0,0]	0%	–
SO synonym rewording	0.05	8.9%	90	0	–	–	0.0 [0,0]	0%	–
SO synonym rewording	0.10	8.9%	90	0	–	–	17.8 [0,55]	77%	0.0%
SO synonym rewording	0.20	8.9%	225	1645	5.81%	6.44 [5.4,7.7]	78.6 [73,82]	100%	0.0%
SO scope widening	0.02	9.4%	90	0	–	–	0.0 [0,0]	0%	–
SO scope widening	0.05	9.4%	765	0	–	–	0.0 [0,0]	0%	–
SO scope widening	0.10	9.4%	1485	361	1.05%	5.26 [3.4,8.1]	34.0 [0,54]	89%	0.0%
SO scope widening	0.20	9.4%	225	1621	4.94%	5.49 [4.5,6.7]	79.4 [72,86]	100%	0.0%
Djinni formatting-only	0.02	12.3%	180	0	–	–	0.0 [0,0]	0%	–
Djinni formatting-only	0.05	12.3%	315	0	–	–	0.0 [0,0]	0%	–
Djinni formatting-only	0.10	12.3%	878	0	–	–	0.0 [0,0]	0%	–
Djinni formatting-only	0.20	12.3%	1617	383	4.17%	8.09 [5.8,11.3]	20.9 [19,28]	100%	0.0%
Djinni criteria change	0.02	12.4%	180	0	–	–	0.0 [0,0]	0%	–
Djinni criteria change	0.05	12.4%	315	0	–	–	0.0 [0,0]	0%	–
Djinni criteria change	0.10	12.4%	918	0	–	–	0.0 [0,0]	0%	–
Djinni criteria change	0.20	12.4%	1442	558	4.88%	6.45 [4.7,8.8]	22.5 [19,28]	100%	0.0%

Matched this way it is a live competitor that wins on volume and loses on where its errors land. At $\alpha=0.2$ on the formatting edit it reused 1,865 cells to sIVM’s 1,724 from the same 135 labels, and at 6.54% realized error it satisfies the bound it was given, against sIVM’s 3.36%. On the widening edit the same pattern: 1,775 reuses at 9.69% versus 1,621 at 5.49%. The extra volume is drawn from the volatile stratum, 180 cached-TRUE cells on the formatting edit and 185 on the widening edit, where the per-stratum floor is 24.1% and no aggregate target can make reuse safe; sIVM reuses none of them. §2 predicts this behaviour: an aggregate target is satisfied by any mixture that averages out, so it buys volume with the correctness of the least stable subgroup, and per-stratum bounds are what extend the guarantee to that part of the table.

With the cached value as the proxy there are only three candidate thresholds, and at $\alpha=0.2$ the aggregate bound clears at the loosest one, so the arm selects 100.0% of the rows it was allowed to on the formatting edit and 100.0% on the widening edit. Matched this way, the competitor reduces to B3, reuse everything, with a certificate attached: its bound is satisfied, its aggregate error is real, and its behaviour is indistinguishable from the guarantee-free status quo. The difference sIVM makes here is that per-stratum bounds refuse a subgroup an aggregate target is willing to spend.

We note two caveats on the same experiment. First, run with an embedding-interaction proxy instead of the cached value the reduction names, and with its much larger threshold search corrected accordingly, the competitor certifies nothing at any budget we tried: a proxy that does not separate flipping rows from stable ones cannot be rescued by the selection procedure wrapped around it, which is what our own stratifier ablation teaches from the other direction. Second, the competitor’s per-test level is looser

than sIVM’s because it runs fewer tests, and its volume advantage is sensitive to that: the comparison is matched on the quantity a user sets (δ in total), not on the level of any individual test, and we report both so the reader can see which one the result rests on.

(B3) *Guarantee-free embedding transfer* (GPTCache-style [1], $\tau=0.95$) reuses every cell on the formatting edit and 84.2% of them on the widening edit, silently absorbing error at roughly the population flip rates (6.3%; 9.45%): the deployed status quo. That τ sits on the shoulder of the widening edit’s collapse, quantified in the threshold sweep below.

Setup and evaluation protocol. All cells use one budget-class reasoning model (DeepSeek V4 Flash, accessed through a single gateway) at temperature zero; every ground-truth pass, sample, and replay is seeded and reproducible from released scripts. Flip labels come from single fresh draws under the new version, which matches the estimand (a label answers “would a recompute differ from the cache now?”). A three-draw relabeling of the headline pair quantifies the instrument’s own variance: with fresh labels on both sides the edit flips 7.2% of cells, versus 6.0% when only the v2 side is drawn fresh against the frozen evaluation vector, and 6.30% under majority-vote labels; 4.8% of rows relabel between instruments. That 6.0% is the same population quantity Table 1 reports as 6.3% for this edit, measured on the same 2,000 rows with a different v2 draw; the two differ by 7 rows, which is what single-draw labelling costs, and we report the churn instead of picking whichever instrument flatters the result. (The within-version floor, 5.0%, is the separate quantity that bounds certifiability.) Labels are individually noisy with about a point of draw-to-draw spread in the aggregate, which the replication standard deviations in Table 1

dominate, and it is population functionals, not labels, that the guarantee addresses. Ledgered model spend across every experiment in this paper is \$11.56 for 98,496 billed cells, read from the system’s own cost ledger. That figure is a lower bound, not a total: the ledger records only cells written through the column runner, and a further 15,416 cells were computed by probe scripts that call the model directly (the self-flip probes, the vote-of-3 study, the multi-draw relabelling, and the second-model replication).

Noise floors and the stability law. Replicate self-runs give the two columns’ intrinsic self-flip rates. The boolean column’s floor is 5.0%, measured from within-version draw pairs over $n=2,000$ rows (an initial $n=200$ two-draw probe read 3.0% and is superseded); the 4-way select’s floor is 6.5% ($n=200$ probe; every observed self-flip was between adjacent tiers). At default temperature the boolean floor rises to 9.5% on that same $n=200$ probe, whose $T=0$ arm read 3.0%: the claim rests on the within-instrument contrast, and the default-temperature floor alone exceeds a 5% budget, which motivates decode pinning. Formatting-only edit flip rates sit above each floor (6.3% over 5.0%; 12.30% over 6.5%): on the boolean column formatting flips are floor-dominated, and on both columns instability scales with task hardness, not edit semantics (the Djinni criteria change lands at 12.45%, indistinguishable from its formatting edit).

A second model family. Every measurement above uses one model, so we replicated the three load-bearing quantities on a second, weaker model from a different family (openai/gpt-5-nano, $n=500$ rows of the same evaluation vector, same prompts, same $T=0$). Its self-flip floor is 36.6%: seven times our primary model’s 5.0%, and above every error budget a user would plausibly set. The formatting edit flips 40.2% of its cells, again barely above its own floor, and the certifier refuses every stratum at $\alpha \in \{0.1, 0.2\}$ after 90 oracle calls. This is the floor prediction of §2 operating as specified: a model that cannot reproduce its own answers cannot support certified reuse at any budget, and the procedure detects that cheaply instead of certifying something unsafe. sIVM’s value is therefore a function of the model’s self-consistency, which is measurable before any edit is made, and the same futility machinery that refuses semantic edits refuses unstable models.

Main results. Table 1 reads as follows. On the stable boolean column, the certifier reuses aggressively where safe (formatting at $\alpha \geq 0.1$; 82.6% savings, interval [82, 86], at $\alpha=0.2$ with realized presented-error 3.20% and a reuse-set rate whose Wilson interval is [2.6, 4.3]) and refuses semantic edits at tight budgets; on the unstable select column, it certifies only at $\alpha=0.2$ and correctly treats formatting and criteria edits alike. Only 9 of the 20 default-mode configurations in Table 1 certify in any replication: budgets below a stratum’s intrinsic rate are refused, and the refusals are the frontier of Figure 2, not failures of the procedure. No unsafe stratum was certified in any seeded run, in 1,000 sampling replications of every configuration, or in the calibration study below; realized presented-cells error stayed within α throughout. One frontier cell shows nonzero reuse-set-metric exceedance, printed in Table 1: conditional on certifying at all, which it does in 0.8% of replications at $\alpha=0.05$, the reuse-set metric exceeds α in 100.0% of those. This does not violate δ , which bounds the certification event, not that metric; nor is it covered by Theorem 1, whose reuse-set guarantee

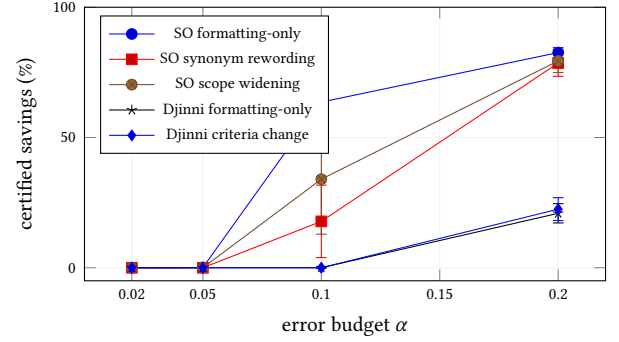


Figure 2: The α -savings frontier under the pinned procedure: mean $\pm$ sd over 1,000 sampling replications per point, generated from the result files. Savings collapse as α approaches each column’s intrinsic flip rate.

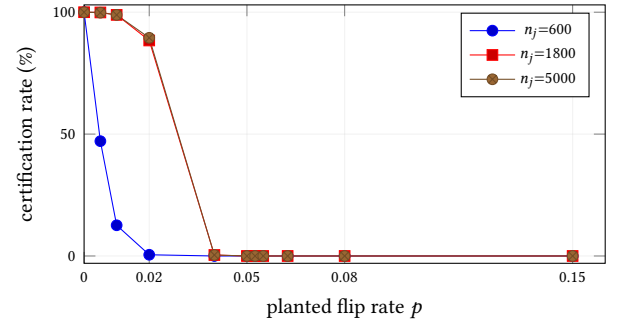


Figure 3: Calibration power at $\alpha=0.05$ (presented estimand, 2,000 trials per point): certification rises smoothly as p falls below α , and is zero for every $p > \alpha$ (no unsafe certifications in any calibration trial). Power saturates once a stratum exceeds what the schedule can sample: the $n_j=1800$ and $n_j=5000$ curves coincide because the six-look schedule caps both at the same sample size.

applies only to certifications made in reuse-set mode, and run in that mode this cell never certifies. The strict reuse-set mode, whose deflated threshold shrinks with sampling depth, prices the stronger guarantee, and the price varies widely across the grid. On the headline edit at $\alpha=0.2$ it is nearly free: 81.4% savings against 82.6%, still certifying in essentially every replication. One budget tighter it collapses, to 8.1% savings certified in 11.7% of replications, and on the unstable select column it never certifies reliably: no Djinni configuration at any budget certifies in more than 0.8% of replications. Guaranteeing the stale subset alone is therefore affordable exactly where the column is stable and the budget is loose, and unaffordable everywhere else.

The α -frontier and adaptive economics. (Figure 2.) Savings collapse as α approaches a stratum’s intrinsic rate, exactly as the power analysis predicts; below the floor the certifier refuses. The doubling schedule prices both directions: uncertifiable edits are refused after 90 oracle calls, whereas a fixed plan pays the

full zero-flip minimum, and near-frontier certifications escalate sampling only while the data stays consistent with certifiability.

Deployment scale. To test the claim that sample cost is table-size-independent, we materialized the boolean column across the *entire* 89,184-row corpus: 78,956 billed cells in a single ledgered run costing \$9.31 at \$0.00012 per cell (the remaining 10,228 rows cost nothing, served by content-hash caching or already materialized by earlier passes), and 7.4 hours of model time reconstructed from the stored per-cell latencies at 32-way concurrency. We then certified the formatting edit against that materialized column with the real model as oracle. Provenance: the materialization and the first certification pass were live; the sweeps reported here are a subsequent replay of the same seeded procedure against the stored cells, which is also what a redundant certification would do in production, so the wall-clock figures below are ledger-derived model-time estimates; the replay itself was not timed. At $\alpha=0.2$ the procedure spent **225 oracle calls** to certify reuse of **81,289 cells**, refusing the 7,715-row cached-TRUE stratum and so recomputing the 7,670 of its cells that the futility stop had not already drawn fresh. That count is one draw from an adaptive schedule, and an unremarkable one: replicating the same procedure over 1,000 resamplings of the benchmark column puts the budget at 208 calls on average with median 225 and range 135–405, so the deployment run landed exactly on that distribution’s median. A deployment should budget for the upper end of that range, not either central value, since the schedule stops on evidence and a near-frontier stratum escalates. Either way the conclusion is unchanged: 91.1% fewer model calls (7,895 calls against 89,184), against a ceiling of 91.3% set by the volatile cached-TRUE stratum that the certifier never certifies: at this budget the procedure captures essentially all of the reuse that is available to capture, and the residue is a property of the column, not of the method. Priced at the ledger’s measured \$0.00012 per cell, that maintenance workload would cost \$0.93 and about 43 minutes of model time, against the \$9.31 and 7.4 hours the corpus actually cost to materialize. We do not quote a decision latency: the only wall clock we hold for these sweeps is the replay’s own (4,257 ms at $\alpha=0.2$), which measures reading stored cells, not issuing the oracle calls a live decision would pay for, and is therefore not a figure any deployment could use. The run also applied the certificate, persisting a `reuse_certificate` row and stamping its identifier on 79,130 of the 81,289 reused cells, which is what the table then serves and what the interface marks as reused. The 2,159 it does not stamp are exactly the audited rows already recomputed for ground truth, because the apply step refuses to write over a cell that already holds a freshly computed value at the new version: a certificate can add a reused value but can never revert a recomputation. We price the recompute leg without executing it: the certifier identified those 7,670 cells, and we did not spend a second corpus materialization recomputing them. At $\alpha=0.1$ the schedule escalated to 765 calls for 90.5% savings.

The comparison to the $n=2,000$ benchmark run shows that sampling cost did not scale with n : at $\alpha=0.2$ the budget was drawn from the same distribution at both scales (135–405 calls, mean 208, against 2,000 rows; 225 against 89,184), because the doubling schedule stops on evidence about a stratum’s *rate*, which is size-independent, not on its size. The budget is not deterministic,

though: a schedule stops at the first look whose bound clears α , so the realized count is a draw, and the two budgets show opposite faces of that. At $\alpha=0.2$ the runs stopped at different looks, 135 calls on the benchmark against 225 at deployment scale: respectively the minimum and the median of the same 135–405 replication distribution, which is the spread an adaptive schedule is supposed to have. At $\alpha=0.1$ they stopped at the *same* look, 765 calls each, which is the maximum of that budget’s replication distribution (mean 568, median 405, range 90–765): a budget close to the stratum’s own flip rate drives many replications to that maximum, and both observed runs landed there. The scaling itself held: sampling cost is $O(1)$ in n while the certifiable population is $O(n)$, so savings rose from 86.2% to 91.1% on the comparable single-run measurement (82.6% is the benchmark’s 1,000-replication mean, quoted elsewhere; mixing the two would attribute part of the gain to an estimator change instead of to scale).

Realized error was audited against oracle-computed v2 labels only (status done/cached; certificate copies are excluded by construction, as verbatim v1 values would register as non-flips): 3.01% over 2,159 audited reused rows, of which 1,807 belong to the released $n=2,000$ evaluation vector and show 3.21% on their own. That subset is most of the audit and carries the same labels as the benchmark row of Table 1, so it is a consistency check, not independent evidence; the genuinely independent portion is the remainder, audited against oracle cells outside the released vector. A separate verifier script recomputes this audit from the run’s persisted reuse-set row identifiers and reproduces both figures exactly. Both are inside both budgets. One reproducibility caveat: in MySQL the projection list changes which rows a seeded `ORDER BY RAND` returns, so the evaluation sample is defined by an exact query string, not by the seed alone.

The stratification hierarchy. Aggregate-only certification (one stratum) is valid for its aggregate estimand yet conceals up to 45.2% realized error in the reused cached-TRUE subgroup: it buys its aggregate with the stratum whose floor is 24.1%, which per-value strata quarantine by construction. Refining further by embedding-interaction tertiles helps only when within-value risk is heterogeneous and otherwise costs power through the δ/K split: on the headline configuration the `valueembedding-tertile` refinement certifies strictly less than value strata alone (61.4% vs 82.6% savings on SO formatting at $\alpha=0.2$), because the split outweighs its discrimination gain. Across all 40 shared configurations of the released ablation the refinement is better in 1 and worse in 16, and the single win is a fraction of a point at a budget where both stratifiers certify essentially nothing. We report this as a negative result instead of tuning until it inverts: the refinement’s validity is unconditional (Assumption 2), its power is not, and on these columns within-value risk is homogeneous enough that the δ/K split is never repaid.

Minority-stratum anatomy. The cached-TRUE stratum of the boolean column flips 37.1% under a formatting edit; vote-of-3 on both sides reduces this to 24.7% and shows that 20.4% of cached TRUE values were single-draw noise; the flips that survive vote-3 are genuine boundary volatility. Quarantine-and-recompute is

therefore the correct systemic response, and the actionable upstream knob is vote-based computation of boundary-stratum cells at write time.

Calibration. (Figure 3.) With planted flip rates across $p \in [0, 0.25]$, budgets $\alpha \in \{0.05, 0.1, 0.2\}$ (including the one every headline result uses), stratum sizes $\{600, 1800, 5000\}$, both estimands, and 198 configurations (396,000 runs), the procedure never certified any stratum with $p > \alpha$, and the realized presented-cells error never exceeded α in any certified run of either mode. 54 of those configurations are *tight* nulls, planted just above the budget at 1.05 to 1.25 times α , where the futility rule cannot dispose of the stratum at the first look and the sampling fraction climbs; the worst certification rate over all of them is 0.0%. This is the regime that probes Assumption 3, since a without-replacement correction would matter only where the bound is close to the truth. The reuse-set rate evaluated on presented-mode certifications does fluctuate past α in rarely-certifying frontier configurations (worst 33.3% of certified runs at $p=0.04$, $\alpha=0.05$, certified 0.45% of the time): this is expected, since presented-mode does not certify that metric; running the strict reuse-set mode drives it to zero everywhere, with certification power rising smoothly as p falls below α and with stratum size. The conservatism gap (realized violations far below nominal δ) is the known cost of Bonferroni composition; it is not budget that can be reclaimed.

The bound is the binding constraint, not the procedure. Every result above spends its budget through a Maurer–Pontil bound split across looks by Bonferroni. That choice, not the stratification or the schedule, is what sets the frontier, and we measured how much by swapping the bound inside the same pinned procedure over the same stored labels. On a clean sample at $n=90$ the Maurer–Pontil bound reads 0.144; a Waudby-Smith–Ramdas betting confidence sequence, which is anytime-valid and therefore spends the stratum’s budget without splitting it across the six looks, reads 0.065; the empirical-Bernstein–Serfling without-replacement bound reads 0.166. End to end across the 20 configurations of Table 1, the betting sequence certifies 9 of them against 7 in the main seeded run (the count of 9 above admits any configuration that certifies in at least one replication), raises mean savings from 18.5% to 29.1%, and does it on 7,616 oracle calls against 11,515: more certificates, more savings, and a third fewer calls, simultaneously. The widest single gap is the scope-widening edit at $\alpha=0.1$, which goes from 18.1% savings for 1,485 calls to 72.0% for 405. The without-replacement bound certifies 8 configurations, on par with Maurer–Pontil: discharging Assumption 3 is roughly free, but gains none of the sequence’s power.

A tighter bound is usable only if it remains valid, so we check validity first. Replaying the calibration protocol with the betting sequence substituted, over 72,000 trials, no null configuration’s certification rate is demonstrably above δ : the worst is 2.5% against a per-stratum δ of 0.05, and 0 of 18 null configurations have a 95% CI lying entirely above it. Maurer–Pontil certifies no null configuration in that same replay: the conservative bound leaves almost all of its error budget unspent, and a clean stratum costs it 360 draws where the sequence needs 180. We report this as an ablation and leave the procedure pinned as run, because the headline results and the deployment run were certified under Maurer–Pontil, and

restating every number under a bound the paper did not run would misattribute the deployment evidence. The frontier in Figure 2 is therefore a property of the bound, and implementations of sIVM should adopt the betting sequence.

Embedding-transfer operating points. A measured τ sweep over per-row similarities shows the guarantee-free transfer baseline collapses within a ≈ 0.02 -wide threshold window (formatting: 100% of rows reused at $\tau=0.96$, 10.4% at 0.98, none at 0.99; widening: 99.9% at 0.94, 8.2% at 0.96, none at 0.97), and, decisively, realized error among reused rows stays at the population flip rate at every operating point (4.8%–6.3% and 8.0%–9.9%); the threshold controls volume, never error, because cross-version similarity carries no per-cell flip signal (B1).

6 The prompt-edit benchmark

Semantic benchmarks exist, but they benchmark *engines* on a fixed task [13]; none exercises maintenance across prompt versions, which needs a different artifact: version pairs plus per-cell flip labels against the new version. Ours comprises: two redistributable corpora (Stack Overflow Developer Survey, 89,184 rows, ODbL 1.0/DbCL 1.0; Djinni English candidate profiles, 210,250 rows, MIT); three materialized columns of two types (two boolean columns over structured facets, one of them the production column and one a lab replica seeded from its second version; one 4-way ordinal select over long free text); five realistic edit pairs spanning formatting-only, synonym rewording, scope widening, and criteria change; full ground-truth flip labels for $n=2,000$ rows per version pair at pinned decode (32,000+ labeled cells), plus self-flip replicate labels for both columns; and the deterministic harness (seeded sampling, adaptive certifier, baseline policies) that reproduces every number in this paper. Ledgered model spend across the entire evaluation is \$11.56 (plus 15,416 cells from direct-call probes the ledger does not see), so the benchmark is cheap to extend to new models.

7 Related work

Guaranteed semantic query processing. Semantic-operator systems bound the error of *one-shot* execution against a gold model: LOTUS’s optimized operators with accuracy targets [20], SUPG’s proxy-based selection with precision/recall guarantees [11], BARGAIN’s adaptive cascade thresholds [26], ThalamusDB’s approximate NL predicates with error bounds [10], and pipeline-level accuracy budgeting in Task Cascades [23]. All certify a *fixed* definition per query; none transfers evidence across a definition change. We reuse their statistical machinery (sampling, EB bounds, cascade framing) in a maintenance loop those systems do not model.

Verified caching. vCache certifies that a *new query* may reuse a cached answer under one fixed definition, with learned per-entry thresholds and user error rates [21]; Evergreen spends confidence sequences and early stopping to cut the cost of *verifying* semantic aggregates, not to decide reuse [14]; and FreshCache approves a cache hit only when a learned staleness model puts the probability that the entry is stale below a per-tier error budget [16]. FreshCache is the closest in shape, since a per-partition budget gating reuse is exactly our skeleton, and it differs on the two axes that

decide the guarantee: its risk is *change-of-world* (open-web evidence ageing under fixed query semantics) where ours is *change-of-definition*, and its per-tier quantity is a fitted model’s point estimate where ours is a finite-sample upper confidence bound that either clears α or refuses. Our B1 baseline shows why these mechanisms do not port: across a definition change the query-to-entry similarity that such caches threshold carries no per-cell flip signal (AUC 0.468 and 0.499). sIVM certifies *change-of-definition* transfer, the axis those systems hold fixed; the two compose (a certificate is scoped to a snapshot, and change-of-world machinery decides when the snapshot itself expires).

Incremental and adaptive view maintenance. Sampling a stale materialized view to bound the error of answers served from it [12] is the closest classical relative of what we do, and the difference is what goes stale: there the data drifts under a fixed view definition, here the definition itself changes over fixed data, which is why their staleness estimators condition on updates and ours condition on a prompt delta. Classical IVM and view adaptation after redefinition [4, 9] assume deterministic operators, where a rewrite either preserves a subexpression or does not. Stochastic LLM cells break the premise: our measurements show that even a semantically null rewrite flips cells at a rate above the column’s noise floor (6.3% against 5.0%), so “preserved” is not a property a rewrite can have here, only a rate one can bound. Streaming semantic systems recompile operators mid-stream but recompute outputs instead of certifying reuse of old ones [6, 7], and streaming cascades certify on the *data-arrival* axis, where new tuples meet a fixed operator, and leave the definition axis open [15]. Given that distinction, sIVM is, to our knowledge, the first maintenance procedure whose unit of transfer is a *statistically certified set of stochastic cells*.

The nearest counterexample shape is step-level reuse with lightweight verification and selective patching across perturbed requests [19], which does reuse computation when a request changes. It differs on exactly the property this paper is about: its verification is a deterministic per-step check, so it carries no error budget and makes no population statement about the set it reuses. In-database prompt management [17] and versioned, provenance-tracked prompt views [5] supply the substrate this work assumes, not a competing mechanism; we take the framing of prompts as versioned views from that line and claim only the certification machinery built on top of it, whose reuse is of *outputs*, not of KV state.

Prompt sensitivity and regression. Prompt-delta analysis for assertion synthesis [22] anticipates our edit taxonomy but targets pipeline quality checks, not reuse certification; interactive LLM regression tools frame migration pain without guarantees [8]. Our finding that surface edit class does not predict flip rate (synonym $\approx$ semantic; formatting $\approx$ floor-dominated) shows why measurement, not taxonomy, must carry the decision.

8 Limitations and outlook

Bonferroni across strata and looks is sound but conservative, and \$5 prices that conservatism instead of asserting it: a betting confidence sequence lifts mean certified savings from 18.5% to 29.1% on a third fewer oracle calls, at a null certification rate that stays inside

δ . Adopting it is the largest power upgrade we measured; e-value selection (e-BH) [24] composes with it. It is not a soundness upgrade: the betting construction is i.i.d., so the without-replacement gap in Assumption 3 stays open, and the Serfling-corrected bound that closes it performs on par with Maurer–Pontil on this grid, so soundness and the sequence’s power gains do not yet combine. Our stratifiers are first instances; the lexical variant is weak on broad edits, and the embedding variant’s signal exists only for semantic edits, so learned interaction models are an open lever with guaranteed validity (Assumption 2). Equivalence is exact-match; free-text columns need a judge whose own error must be calibrated and folded into α . Measurements use one budget-class reasoning model for the full experimental program and a second, weaker family only for the floor and refusal check; the quantitative frontier (where savings sit for a given α) is therefore model-specific, even though the qualitative structure, a measurable floor that bounds certifiability, held on both. Certificates are scoped to pinned snapshots and say nothing about provider drift; composing sIVM with change-of-world invalidation [14, 16] is natural future work. Finally, the economic frame that matters in production is time-to-consistency under provider rate limits, where certified maintenance samples 7%–81% of cells across the reliably-certifying configurations of Table 1 (7% on the headline formatting edit at $\alpha=0.2$) instead of re-computing 100%, a wall-clock and rate-limit gain that grows with table size, because sample sizes do not.

9 Conclusion

Prompt-defined columns are materialized views whose *definition* changes far more often than their data, and the systems serving them today choose between recomputing everything and reusing everything. sIVM shows the middle is reachable and cheap: freeze a stratification, sample on a doubling schedule, bound each stratum’s flip rate, reuse only what clears the budget. At deployment scale that cost 225 oracle calls to certify reuse of 81,289 of 89,184 cells, and the sample did not have to grow with the table, because the schedule stops on evidence about a *rate*, which is size-independent.

The result we did not anticipate is that the certifier doubles as a measuring instrument for the column it maintains. A stratum refused at every budget is a slice of the table on which the model cannot reproduce its own answers, identified by a futility stop at the cost of one look. Certified maintenance is therefore useful even where it certifies nothing: the refusal identifies, cheaply, reuse error that would otherwise go unmeasured.

Availability and reproducibility

The system, the certifier, every experiment script, and every result artifact are prepared for deposit as a single archive; the archive URL will be added to this paper when it is public. Released: the two corpora (already public and redistributable under ODbL 1.0/DbCL 1.0 and MIT), the five versioned edit pairs, the per-cell ground-truth flip labels, the certifier, the adapter conformance suite, and every script and result artifact that produces a number in this paper. Withheld until the archive is public: nothing in the evaluation path. Anyone reproducing this needs a model endpoint of their own; the labels are released so the statistical results can be reproduced without one. Several numbers here

(the per-stratum floors, the ledger economics, the materialization wall clock, the certificate’s stamp count) are computed from the running system’s database, not from a result file, so the generator dumps every one of those database reads to an artifact alongside the others; the release is therefore self-contained, with no dependence on our instance. Every quantitative claim in this paper’s prose, tables, and figures is emitted by a single generator (scripts/experiments/gen-paper-assets.ts) from those artifacts and from the system’s cost ledger, and cited through macros, never typed inline, so re-running an experiment regenerates the sentences that cite it. This is enforced mechanically: scripts/check-paper-numbers.ts fails on any of nine conditions: (1) a hand-typed quantity in the prose outside an enumerated allow-list (user parameters such as α and δ , protocol constants such as the schedule looks, and generic thresholds quoted as such); (2) a quantity hardcoded inside the generator instead of derived from an artifact; (3) a macro defined but never cited, since an uncited macro can drift undetected; (4) a macro whose trailing space is swallowed into the next word, which is invisible in the source; (5) a sentence that asserts two quantities differ while citing two macros whose values are equal, which is how an earlier draft printed “different looks” beside a number repeated twice; (6) a spelled-out multiplier (“twice”, “an order of magnitude”) that the cited macros’ ratio does not support, which is how a rate 1.26 times a floor was printed as double it; (7) a containment claim (“inside”, “at most”) that the cited range and scalar macros do not support; (8) an absolute claim (“never”, “every”, “anywhere”) appearing in the abstract or the contributions list without an entry in an enumerated allow-list, the one class of defect the numeric guards are blind to by construction; and (9) a scope claim (“on the same column”) whose cited macros come from different columns, checked against a provenance file the generator emits from the same pair definitions the experiments run on. It reports zero of all nine for this draft. Each guard exists because the corresponding defect reached a compiled PDF at least once, several of them in a draft whose numbers were already machine-generated: generating the quantities is necessary but not sufficient, because the sentences that *relate* quantities go stale independently of the quantities themselves. Two reproducibility hazards we hit are documented in the harness: seeded ORDER BY RAND in MySQL depends on the projection list, so evaluation samples are pinned by exact query string; and naive linear-congruential PRNGs written with JavaScript number multiplication silently lose low bits, so all sampling uses exact 32-bit arithmetic.

References

- [1] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications. In *Proc. Workshop for Natural Language Processing Open Source Software (NLP-OSS) at EMNLP*.
- [2] Rémi Bardenet and Odalric-Ambrym Maillard. 2015. Concentration Inequalities for Sampling without Replacement. *Bernoulli* 21, 3 (2015).
- [3] Abhinaba Basu. 2026. Cross-Domain Uncertainty Quantification for Selective Prediction: A Comprehensive Bound Ablation with Transfer-Informed Betting. arXiv:2603.08907
- [4] Mihai Budiu, Frank McSherry, Leonid Ryzhyk, and Val Tannen. 2023. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *Proc. VLDB Endow.* 16, 7 (2023), 1601–1614.
- [5] Uğur Çetintemel, Shu Chen, Alexander W. Lee, Deepti Raghavan, Duo Lu, and Andrew Crotty. 2026. Making Prompts First-Class Citizens for Adaptive LLM Pipelines. In *Conf. on Innovative Data Systems Research (CIDR)*. arXiv:2508.05012.
- [6] Shu Chen, Junhan Liu, Deepti Raghavan, and Uğur Çetintemel. 2026. VectraFlow: Long-Horizon Semantic Processing over Data and Event Streams with LLMs. arXiv:2604.03855
- [7] Shu Chen, Deepti Raghavan, and Uğur Çetintemel. 2025. Continuous Prompts: LLM-Augmented Pipeline Processing over Unstructured Streams. arXiv:2512.03389
- [8] Tanay Dixit, Daniel Lee, Sally Fang, Sai Sree Harsha, Anirudh Sureshan, Akash Maharaj, and Yunyao Li. 2024. RETAIN: Interactive Tool for Regression Testing Guided LLM Migration. In *Proc. Conf. on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*. 301–310. arXiv:2409.03928.
- [9] Ashish Gupta, Inderpal Singh Mumick, and Kenneth A. Ross. 1995. Adapting Materialized Views after Redefinitions. In *Proc. ACM SIGMOD Int. Conf. on Management of Data*.
- [10] Saehan Jo and Immanuel Trummer. 2024. ThalamusDB: Approximate Query Processing on Multi-Modal Data. *Proc. ACM Manag. Data* 2, 3 (2024), 1–26. <https://doi.org/10.1145/3654989>
- [11] Daniel Kang, Edward Gan, Peter Bailis, Tatsunori Hashimoto, and Matei Zaharia. 2020. Approximate Selection with Guarantees Using Proxies. *Proc. VLDB Endow.* 13, 11 (2020), 1990–2003.
- [12] Sanjay Krishnan, Jiannan Wang, Michael J. Franklin, Ken Goldberg, and Tim Kraska. 2015. Stale View Cleaning: Getting Fresh Answers from Stale Materialized Views. *Proc. VLDB Endow.* 8, 12 (2015), 1370–1381.
- [13] Jiale Lao et al. 2026. SemBench: A Benchmark for Semantic Query Processing Engines. arXiv:2511.01716 VLDB 2026.
- [14] Alexander W. Lee, Benjamin Han, Shayak Sen, Sam Yeom, Uğur Çetintemel, and Anupam Datta. 2026. Evergreen: Efficient Claim Verification for Semantic Aggregates. arXiv:2604.26180
- [15] Paweł Liskowski and Kyle Schmaus. 2026. Streaming Model Cascades for Semantic SQL. arXiv:2604.00660
- [16] Muhammad Mansoor, Tahir Ahmad, and Yeo-Chan Yoon. 2026. Risk-Constrained Freshness-Aware Semantic Caching for Open-Web Retrieval-Augmented LLMs. arXiv:2607.04281
- [17] Denis Mayr Lima Martins and Gottfried Vossen. 2026. Prompt as a Data Type: In-Database LLM Prompt Management and Rewriting. arXiv:2607.21756
- [18] Andreas Maurer and Massimiliano Pontil. 2009. Empirical Bernstein Bounds and Sample-Variance Penalization. In *Proc. Conf. on Learning Theory (COLT)*.
- [19] Azam Nouri. 2026. StepCache: Step-Level Reuse with Lightweight Verification and Selective Patching for LLM Serving. arXiv:2603.28795
- [20] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proc. VLDB Endow.* 18, 11 (2025), 4171–4184. <https://doi.org/10.14778/3749646.3749685>
- [21] Luis Gaspar Schroeder, Aditya Desai, Alejandro Cuadron, Kyle Chu, Shu Liu, Mark Zhao, Stephan Krusche, Alfons Kemper, Matei Zaharia, and Joseph E. Gonzalez. 2026. vCache: Verified Semantic Prompt Caching. In *Proc. Int. Conf. on Learning Representations (ICLR)*. arXiv:2502.03771.
- [22] Shreya Shankar, Haotian Li, Parth Asawa, Madelon Hulsebos, Yiming Lin, J. D. Zambrescu-Pereira, Harrison Chase, Will Fu-Hinthorn, Aditya G. Parameswaran, and Eugene Wu. 2024. SPADe: Synthesizing Data Quality Assertions for Large Language Model Pipelines. arXiv:2401.03038
- [23] Shreya Shankar, Sepanta Zeighami, and Aditya Parameswaran. 2026. Task Cascades for Efficient Unstructured Data Processing. *Proc. ACM Manag. Data* (2026). <https://doi.org/10.1145/3786702> SIGMOD 2026.
- [24] Ruodu Wang and Aaditya Ramdas. 2022. False Discovery Rate Control with E-values. *J. R. Stat. Soc. Series B* 84, 3 (2022).
- [25] Ian Waudby-Smith and Aaditya Ramdas. 2024. Estimating Means of Bounded Random Variables by Betting. *J. R. Stat. Soc. Series B* (2024).
- [26] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2026. Cut Costs, Not Accuracy: LLM-Powered Data Processing with Guarantees. *Proc. ACM Manag. Data* (2026). <https://doi.org/10.1145/3769776> SIGMOD 2026.